

Dissertation dev log

Week 18:

Current Goals for Full Dissertation session:

- Research question made more precise and naturalistic
- Workplan make clearer
- Gather data
- Analyse data
- Specification for computing artefact that integrates societal, user, and business needs, demonstrating impact, usability and values to stakeholders
- Design rational, usage guidance, maintainability and architecture overview are provided
- Abstract
- Results and analysis section
- Discussions and conclusion
- Statistics addendum
- Artefact testing
- Critical addendum

Completed:

- A way to interact with the fragmented mesh
- A working Voronoi fracture in 2D for demonstration
- Working Voronoi fracture in 3D
- Comprehensive background done, covering triangulation, fracturing methods used in games, Graphics API's, Performance of Graphics API's, Point sampling, and Voronoi Clipping.
- Clearly stated research question
- Clear, feasible hypothesis, that is well scoped.
- An introduction that provides clear context for the project, and explains the need for the research, including positive impacts and who benefits. It should also highlight the motivation for the project.
- Professional, ethical, sustainable, legal, and social review carried out.
- A plan for security and privacy concerns has been written, and data is well managed.
- A methodology for how the project is going to be researched, and how its going to be evaluated
- A working way of profiling the results of the prototype
- Voronoi clipping implemented

- A Critical Review of current technology is completed, that highlights where the gap in research is, and the how this project fills it.
- Project is ethically robust
- Project plan with gantt chart
- Profiling results that are easily replicated and are well documented.
- Fracture implemented in both OpenGL and Vulkan
- Readme that highlights development roadmap
- OpenGL integrated into bridge system
- An implementation in GLSL, for performance comparison
-

Bugs

- With plane clipping, when using 1000 points, some of the cells seem to randomly grow then shrink in size, could be due to the rendering engine, rather than Voronoi
- OpenGL buffering of data has to be done in a specific order, i.e., declare positions, then buffer it, then declare textures, and then buffer them
- Mesh needs tetrahedralization, which means constrained Delaunay triangulation
- Cannot output to many points due to it being too large. Switch to triangles to reduce size?

Development notes:

A glsl compute shader for Voronoi cell generation, and Voronoi clipping have been implemented. Compute shader dispatching and SSBO's have been implemented in both OpenGL and Vulkan. However, scaling the points up to 1000 causes both to crash, due to too much memory being used. To solve this, I plan to switch to cutting by mesh triangles instead of tetrahedrons. This would decrease output size as instead of needing 1000 faces, 20 should be enough per cell. This would also mean non-constrained meshes could be used, but accuracy could potentially be reduced, as there are edge cases with irregular shapes that would break cell generation. Should be ready to start data collection soon.

Images:

References:

